

```

    fn.resize(nfun);
    sf.assign(nfun,0.);
    sferr.assign(nfun,0.);
    for (Int j=0;j<ndim;j++) vol *= abs(xhi[j]-xlo[j]);
}

void MCIntegrate::step(Int nstep) {
    Int i,j;
    for (i=0;i<nstep;i++) {
        for (j=0;j<ndim;j++)
            x[j] = xlo[j]+(xhi[j]-xlo[j])*ran.doub();
        if (xmapp) xx = (*xmapp)(x);
        else xx = x;
        if ((*inregionp)(xx)) {
            fn = (*funcsp)(xx);
            for (j=0;j<nfun;j++) {
                sf[j] += fn[j];
                sferr[j] += SQR(fn[j]);
            }
        }
    }
    n += nstep;
}

void MCIntegrate::calcanswers(){
    for (Int j=0;j<nfun;j++) {
        ff[j] = vol*sf[j]/n;
        fferr[j] = vol*sqrt((sferr[j]/n-SQR(sf[j]/n))/n);
    }
}

```

CITED REFERENCES AND FURTHER READING:

Robert, C.P., and Casella, G. 2006, *Monte Carlo Statistical Methods*, 2nd ed. (New York: Springer)

Sobol', I.M. 1994, *A Primer for the Monte Carlo Method* (Boca Raton, FL: CRC Press).

Hammersley, J.M., and Handscomb, D.C. 1964, *Monte Carlo Methods* (London: Methuen).

Gentle, J.E. 2003, *Random Number Generation and Monte Carlo Methods*, 2nd ed. (New York: Springer), Chapter 7.

Shreider, Yu. A. (ed.) 1966, *The Monte Carlo Method* (Oxford: Pergamon).

Kalos, M.H., and Whitlock, P.A. 1986, *Monte Carlo Methods: Volume 1: Basics* (New York: Wiley).

7.8 Quasi- (that is, Sub-) Random Sequences

We have just seen that choosing N points uniformly randomly in an n -dimensional space leads to an error term in Monte Carlo integration that decreases as $1/\sqrt{N}$. In essence, each new point sampled adds linearly to an accumulated sum that will become the function average, and also linearly to an accumulated sum of squares that will become the variance (equation 7.7.2). The estimated error comes from the square root of this variance, hence the power $N^{-1/2}$.

Just because this square-root convergence is familiar does not, however, mean that it is inevitable. A simple counterexample is to choose sample points that lie on a Cartesian grid, and to sample each grid point exactly once (in whatever order).

The Monte Carlo method thus becomes a deterministic quadrature scheme — albeit a simple one — whose fractional error decreases at least as fast as N^{-1} (even faster if the function goes to zero smoothly at the boundaries of the sampled region or is periodic in the region).

The trouble with a grid is that one has to decide *in advance* how fine it should be. One is then committed to completing all of its sample points. With a grid, it is not convenient to “sample *until*” some convergence or termination criterion is met. One might ask if there is not some intermediate scheme, some way to pick sample points “at random,” yet spread out in some self-avoiding way, avoiding the chance clustering that occurs with uniformly random points.

A similar question arises for tasks other than Monte Carlo integration. We might want to search an n -dimensional space for a point where some (locally computable) condition holds. Of course, for the task to be computationally meaningful, there had better be continuity, so that the desired condition will hold in some finite n -dimensional neighborhood. We may not know a priori how large that neighborhood is, however. We want to “sample *until*” the desired point is found, moving smoothly to finer scales with increasing samples. Is there any way to do this that is better than uncorrelated, random samples?

The answer to the above question is “yes.” Sequences of n -tuples that fill n -space more uniformly than uncorrelated random points are called *quasi-random sequences*. That term is somewhat of a misnomer, since there is nothing “random” about quasi-random sequences: They are cleverly crafted to be, in fact, *subrandom*. The sample points in a quasi-random sequence are, in a precise sense, “maximally avoiding” of each other.

A conceptually simple example is *Halton’s sequence* [1]. In one dimension, the j th number H_j in the sequence is obtained by the following steps: (i) Write j as a number in base b , where b is some prime. (For example, $j = 17$ in base $b = 3$ is 122.) (ii) Reverse the digits and put a radix point (i.e., a decimal point base b) in front of the sequence. (In the example, we get 0.221 base 3.) The result is H_j . To get a sequence of n -tuples in n -space, you make each component a Halton sequence with a different prime base b . Typically, the first n primes are used.

It is not hard to see how Halton’s sequence works: Every time the number of digits in j increases by one place, j ’s digit-reversed fraction becomes a factor of b finer-meshed. Thus the process is one of filling in all the points on a sequence of finer and finer Cartesian grids — and in a kind of maximally spread-out order on each grid (since, e.g., the most rapidly changing digit in j controls the *most* significant digit of the fraction).

Other ways of generating quasi-random sequences have been proposed by Faure, Sobol’, Niederreiter, and others. Bratley and Fox [2] provide a good review and references, and discuss a particularly efficient variant of the Sobol’ [3] sequence suggested by Antonov and Saleev [4]. It is this Antonov-Saleev variant whose implementation we now discuss.

The Sobol’ sequence generates numbers between zero and one directly as binary fractions of length w bits, from a set of w special binary fractions, V_i , $i = 1, 2, \dots, w$, called *direction numbers*. In Sobol’ original method, the j th number X_j is generated by XORing (bitwise exclusive or) together the set of V_i ’s satisfying the criterion on i , “the i th bit of j is nonzero.” As j increments, in other words, different ones of the V_i ’s flash in and out of X_j on different time scales. V_1 alternates between being present and absent most quickly, while

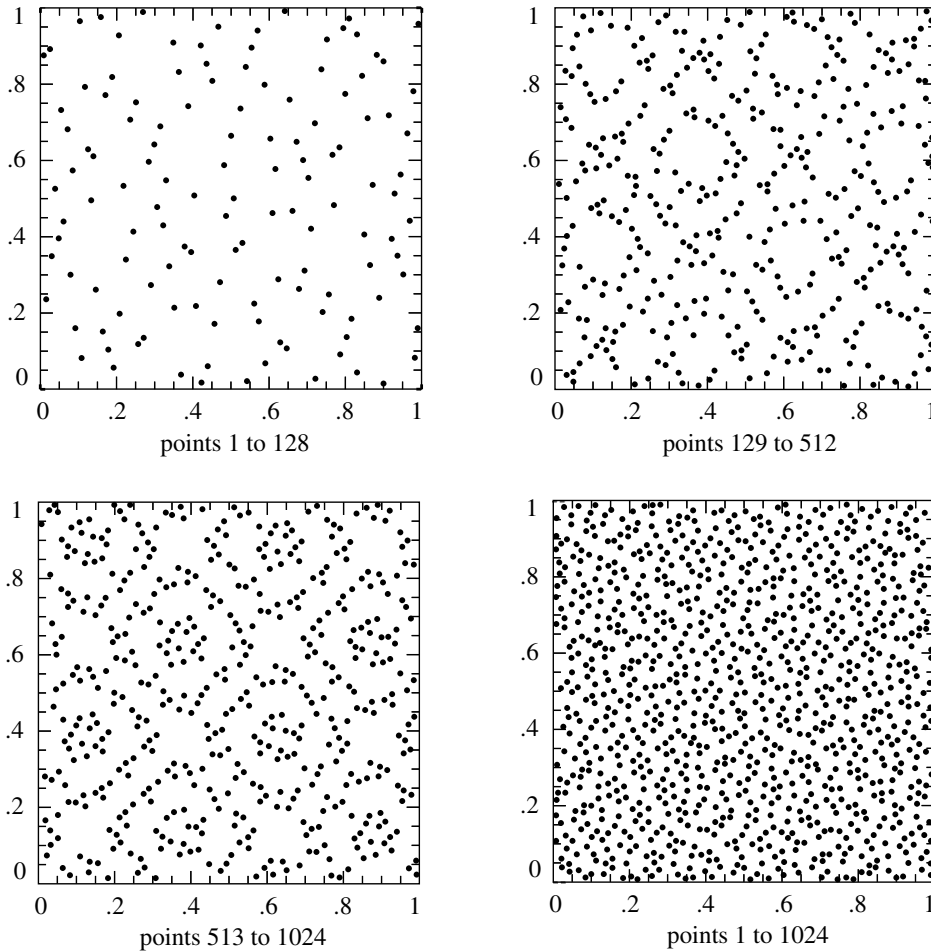


Figure 7.8.1. First 1024 points of a two-dimensional Sobol' sequence. The sequence is generated number-theoretically, rather than randomly, so successive points at any stage “know” how to fill in the gaps in the previously generated distribution.

V_k goes from present to absent (or vice versa) only every 2^{k-1} steps.

Antonov and Saleev's contribution was to show that instead of using the bits of the integer j to select direction numbers, one could just as well use the bits of the *Gray code* of j , $G(j)$. (For a quick review of Gray codes, look at §22.3.)

Now $G(j)$ and $G(j+1)$ differ in exactly one bit position, namely in the position of the rightmost zero bit in the binary representation of j (adding a leading zero to j if necessary). A consequence is that the $j+1$ st Sobol'-Antonov-Saleev number can be obtained from the j th by XORing it with a *single* V_i , namely with i the position of the rightmost zero bit in j . This makes the calculation of the sequence very efficient, as we shall see.

Figure 7.8.1 plots the first 1024 points generated by a two-dimensional Sobol' sequence. One sees that successive points do “know” about the gaps left previously, and keep filling them in, hierarchically.

We have deferred to this point a discussion of how the direction numbers V_i are generated. Some nontrivial mathematics is involved in that, so we will content ourselves with a cookbook summary only: Each different Sobol' sequence (or component of an n -dimensional

Degree	Primitive Polynomials Modulo 2*
1	0 (i.e., $x + 1$)
2	1 (i.e., $x^2 + x + 1$)
3	1, 2 (i.e., $x^3 + x + 1$ and $x^3 + x^2 + 1$)
4	1, 4 (i.e., $x^4 + x + 1$ and $x^4 + x^3 + 1$)
5	2, 4, 7, 11, 13, 14
6	1, 13, 16, 19, 22, 25
7	1, 4, 7, 8, 14, 19, 21, 28, 31, 32, 37, 41, 42, 50, 55, 56, 59, 62
8	14, 21, 22, 38, 47, 49, 50, 52, 56, 67, 70, 84, 97, 103, 115, 122
9	8, 13, 16, 22, 25, 44, 47, 52, 55, 59, 62, 67, 74, 81, 82, 87, 91, 94, 103, 104, 109, 122, 124, 137, 138, 143, 145, 152, 157, 167, 173, 176, 181, 182, 185, 191, 194, 199, 218, 220, 227, 229, 230, 234, 236, 241, 244, 253
10	4, 13, 19, 22, 50, 55, 64, 69, 98, 107, 115, 121, 127, 134, 140, 145, 152, 158, 161, 171, 181, 194, 199, 203, 208, 227, 242, 251, 253, 265, 266, 274, 283, 289, 295, 301, 316, 319, 324, 346, 352, 361, 367, 382, 395, 398, 400, 412, 419, 422, 426, 428, 433, 446, 454, 457, 472, 493, 505, 508
*Expressed as a decimal integer whose binary representation gives the coefficients, from the highest to lowest power of x . Only the internal terms are represented — the highest-order term and the constant term always have coefficient 1.	

sequence) is based on a different primitive polynomial over the integers modulo 2, that is, a polynomial whose coefficients are either 0 or 1, and which generates a maximal length shift register sequence. (Primitive polynomials modulo 2 were used in §7.5 and are further discussed in §22.4.) Suppose P is such a polynomial, of degree q ,

$$P = x^q + a_1 x^{q-1} + a_2 x^{q-2} + \cdots + a_{q-1} x + 1 \quad (7.8.1)$$

Define a sequence of integers M_i by the q -term recurrence relation,

$$M_i = 2a_1 M_{i-1} \oplus 2^2 a_2 M_{i-2} \oplus \cdots \oplus 2^{q-1} M_{i-q+1} a_{q-1} \oplus (2^q M_{i-q} \oplus M_{i-q}) \quad (7.8.2)$$

Here bitwise XOR is denoted by $\oplus$. The starting values for this recurrence are that $M_1, \dots, M_q$ can be arbitrary odd integers less than $2, \dots, 2^q$, respectively. Then, the direction numbers V_i are given by

$$V_i = M_i / 2^i \quad i = 1, \dots, w \quad (7.8.3)$$

The table above lists all primitive polynomials modulo 2 with degree $q \leq 10$. Since the coefficients are either 0 or 1, and since the coefficients of x^q and of 1 are predictably 1, it is convenient to denote a polynomial by its middle coefficients taken as the bits of a binary number (higher powers of x being more significant bits). The table uses this convention.

Turn now to the implementation of the Sobol' sequence. Successive calls to the function `sobseq` (after a preliminary initializing call) return successive points in an n -dimensional Sobol' sequence based on the first n primitive polynomials in the table. As given, the routine is initialized for maximum n of 6 dimensions, and for a word length w of 30 bits. These parameters can be altered by changing `MAXBIT` ($\equiv w$) and `MAXDIM`, and by adding more initializing data to the arrays `ip` (the primitive polynomials from the table above), `mdeg` (their degrees), and `iv` (the starting values for the recurrence, equation 7.8.2). A second table, on the next page, elucidates the initializing data in the routine.

Initializing Values Used in sobseq					
Degree	Polynomial	Starting Values			
1	0	1	(3)	(5)	(15)...
2	1	1	1	(7)	(11)...
3	1	1	3	7	(5)...
3	2	1	3	3	(15)...
4	1	1	1	3	13...
4	4	1	1	5	9...
Parenthesized values are not freely specifiable, but are forced by the required recurrence for this degree.					

```
void sobseq(const Int n, VecDoub_0 &x)
```

sobseq.h

When n is negative, internally initializes a set of MAXBIT direction numbers for each of MAXDIM different Sobol' sequences. When n is positive (but $\leq \text{MAXDIM}$), returns as the vector $x[0..n-1]$ the next values from n of these sequences. (n must not be changed between initializations.)

```
{
    const Int MAXBIT=30,MAXDIM=6;
    Int j,k,l;
    Uint i,im,ipp;
    static Int mdeg[MAXDIM]={1,2,3,3,4,4};
    static Uint in;
    static VecUint ix(MAXDIM);
    static NRvector<Uint*> iu(MAXBIT);
    static Uint ip[MAXDIM]={0,1,1,2,1,4};
    static Uint iv[MAXDIM*MAXBIT]=
        {1,1,1,1,1,1,3,1,3,3,1,1,5,7,7,3,3,5,15,11,5,15,13,9};
    static Doub fac;

    if (n < 0) {
        for (k=0;k<MAXDIM;k++) ix[k]=0;
        in=0;
        if (iv[0] != 1) return;
        fac=1.0/(1 << MAXBIT);
        for (j=0,k=0;j<MAXBIT;j++,k+=MAXDIM) iu[j] = &iv[k];
        To allow both 1D and 2D addressing.
        for (k=0;k<MAXDIM;k++) {
            for (j=0;j<mdeg[k];j++) iu[j][k] <<= (MAXBIT-1-j);
            Stored values only require normalization.
            for (j=mdeg[k];j<MAXBIT;j++) {
                ipp=ip[k];
                i=iu[j-mdeg[k]][k];
                i ^= (i >> mdeg[k]);
                for (l=mdeg[k]-1;l>=1;l--) {
                    if (ipp & 1) i ^= iu[j-1][k];
                    ipp >>= 1;
                }
                iu[j][k]=i;
            }
        }
    } else {
        im=in++;
        for (j=0;j<MAXBIT;j++) {
            if (!(im & 1)) break;
            im >>= 1;
        }
    }
}
```

Initialize, don't return a vector.

Use the recurrence to get other values.

Calculate the next vector in the sequence.
Find the rightmost zero bit.

```

if (j >= MAXBIT) throw("MAXBIT too small in sobseq");
im=j*MAXDIM;
for (k=0;k<MIN(n,MAXDIM);k++) {
    ix[k] ^= iv[im+k];
    x[k]=ix[k]*fac;
}
}
}

```

XOR the appropriate direction number into each component of the vector and convert to a floating number.

How good is a Sobol' sequence, anyway? For Monte Carlo integration of a smooth function in n dimensions, the answer is that the fractional error will decrease with N , the number of samples, as $(\ln N)^n/N$, i.e., almost as fast as $1/N$. As an example, let us integrate a function that is nonzero inside a torus (doughnut) in three-dimensional space. If the major radius of the torus is R_0 and the minor radius is r_0 , the minor radial coordinate r is defined by

$$r = \left((x^2 + y^2)^{1/2} - R_0 \right)^2 + z^2 \Big)^{1/2} \quad (7.8.4)$$

Let us try the function

$$f(x, y, z) = \begin{cases} 1 + \cos\left(\frac{\pi r^2}{r_0^2}\right) & r < r_0 \\ 0 & r \geq r_0 \end{cases} \quad (7.8.5)$$

which can be integrated analytically in cylindrical coordinates, giving

$$\iiint dx dy dz f(x, y, z) = 2\pi^2 r_0^2 R_0 \quad (7.8.6)$$

With parameters $R_0 = 0.6$, $r_0 = 0.3$, we did 100 successive Monte Carlo integrations of equation (7.8.4), sampling uniformly in the region $-1 < x, y, z < 1$, for the two cases of uncorrelated random points and the Sobol' sequence generated by the routine `sobseq`. Figure 7.8.2 shows the results, plotting the r.m.s. average error of the 100 integrations as a function of the number of points sampled. (For any *single* integration, the error of course wanders from positive to negative, or vice versa, so a logarithmic plot of fractional error is not very informative.) The thin, dashed curve corresponds to uncorrelated random points and shows the familiar $N^{-1/2}$ asymptotics. The thin, solid gray curve shows the result for the Sobol' sequence. The logarithmic term in the expected $(\ln N)^3/N$ is readily apparent as curvature in the curve, but the asymptotic N^{-1} is unmistakable.

To understand the importance of Figure 7.8.2, suppose that a Monte Carlo integration of f with 1% accuracy is desired. The Sobol' sequence achieves this accuracy in a few thousand samples, while pseudo-random sampling requires nearly 100,000 samples. The ratio would be even greater for higher desired accuracies.

A different, not quite so favorable, case occurs when the function being integrated has hard (discontinuous) boundaries inside the sampling region, for example the function that is one inside the torus and zero outside,

$$f(x, y, z) = \begin{cases} 1 & r < r_0 \\ 0 & r \geq r_0 \end{cases} \quad (7.8.7)$$

where r is defined in equation (7.8.4). Not by coincidence, this function has the same analytic integral as the function of equation (7.8.5), namely $2\pi^2 r_0^2 R_0$.

The carefully hierarchical Sobol' sequence is based on a set of Cartesian grids, but the boundary of the torus has no particular relation to those grids. The result is that it is essentially random whether sampled points in a thin layer at the surface of the torus, containing on the order of $N^{2/3}$ points, come out to be inside or outside the torus. The square root law, applied to this thin layer, gives $N^{1/3}$ fluctuations in the sum, or $N^{-2/3}$ fractional error in the Monte Carlo integral. One sees this behavior verified in Figure 7.8.2 by the thicker gray curve. The

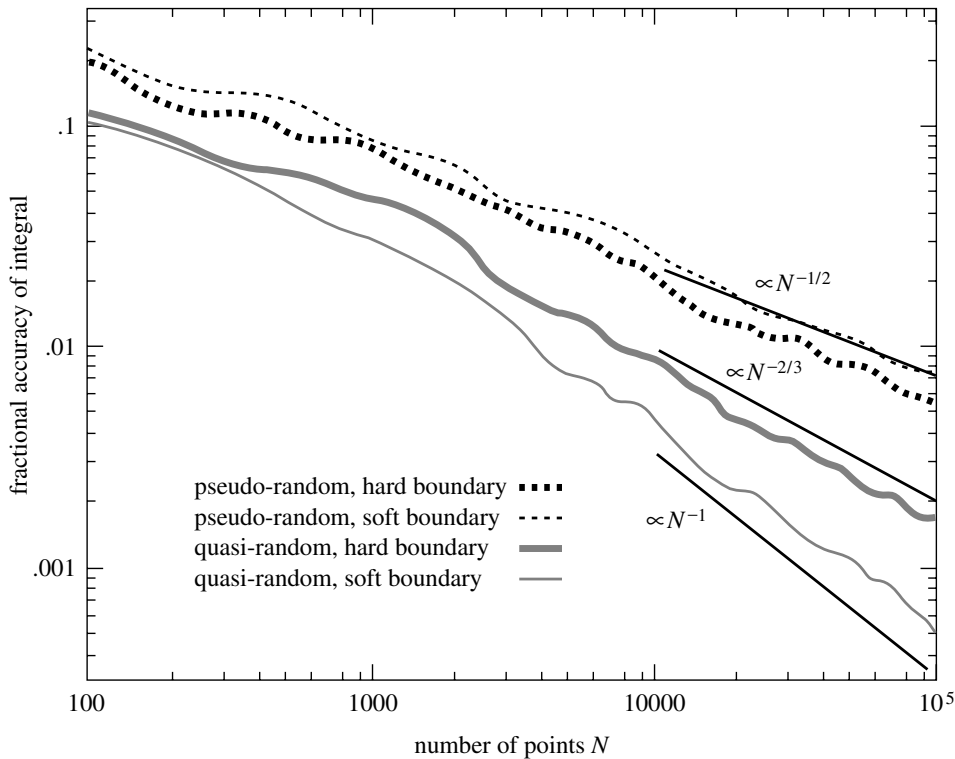


Figure 7.8.2. Fractional accuracy of Monte Carlo integrations as a function of number of points sampled, for two different integrands and two different methods of choosing random points. The quasi-random Sobol’ sequence converges much more rapidly than a conventional pseudo-random sequence. Quasi-random sampling does better when the integrand is smooth (“soft boundary”) than when it has step discontinuities (“hard boundary”). The curves shown are the r.m.s. averages of 100 trials.

thicker dashed curve in Figure 7.8.2 is the result of integrating the function of equation (7.8.7) using independent random points. While the advantage of the Sobol’ sequence is not quite so dramatic as in the case of a smooth function, it can nonetheless be a significant factor (~ 5) even at modest accuracies like 1%, and greater at higher accuracies.

Note that we have not provided the routine `sobseq` with a means of starting the sequence at a point other than the beginning, but this feature would be easy to add. Once the initialization of the direction numbers i_v has been done, the j th point can be obtained directly by XORing together those direction numbers corresponding to nonzero bits in the Gray code of j , as described above.

7.8.1 The Latin Hypercube

We mention here the unrelated technique of *Latin square* or *Latin hypercube* sampling, which is useful when you must sample an N -dimensional space *exceedingly* sparsely, at M points. For example, you may want to test the crashworthiness of cars as a simultaneous function of four different design parameters, but with a budget of only three expendable cars. (The issue is not whether this is a good plan — it isn’t — but rather how to make the best of the situation!)

The idea is to partition each design parameter (dimension) into M segments, so that the whole space is partitioned into M^N cells. (You can choose the segments